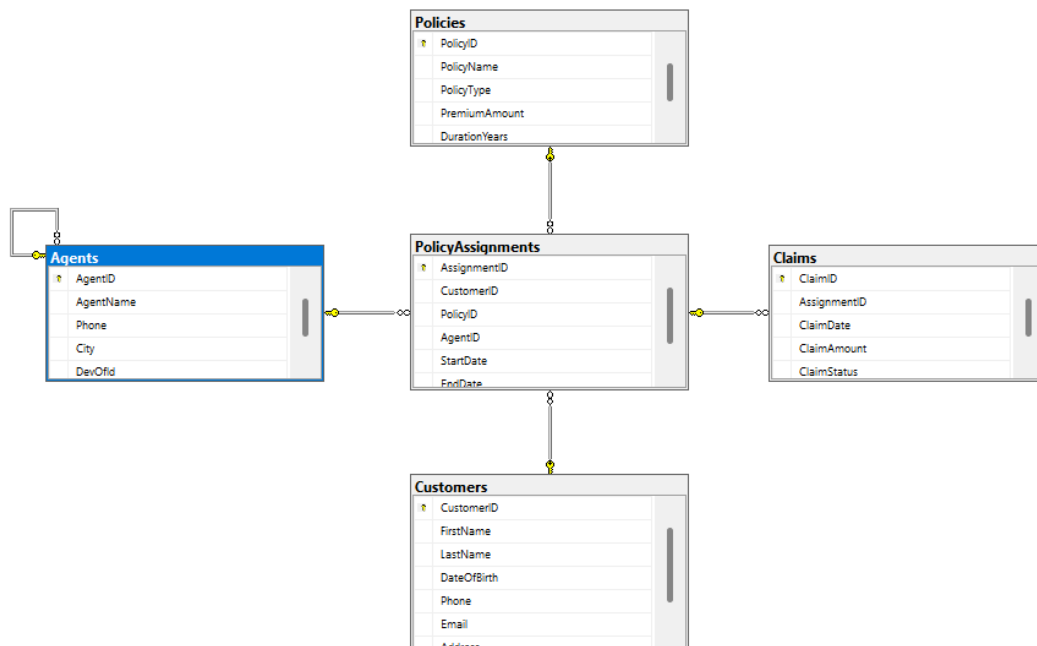


MODULE 4.4 PRACTICAL PROJECT ASSIGNMENT

1. Data Base Creation:

```
create database InsuranceDB;  
use InsuranceDB;
```

2. Schema Diagram:



3. Tables Creation :

```
CREATE TABLE Customers (  
    CustomerID INT PRIMARY KEY,  
    FirstName VARCHAR(50) NOT NULL,  
    LastName VARCHAR(50),  
    DateOfBirth DATE,  
    Phone VARCHAR(15),  
    Email VARCHAR(50)  
);
```

```
CREATE TABLE Policies (  
    PolicyID INT PRIMARY KEY,  
    PolicyName VARCHAR(50),  
    PolicyType VARCHAR(50),  
    PremiumAmount MONEY,  
    DurationYears INT  
);
```

```
CREATE TABLE Agents (  
    AgentID INT PRIMARY KEY,  
    AgentName VARCHAR(50),  
    Phone VARCHAR(15),  
    City VARCHAR(20)
```

```

);

CREATE TABLE PolicyAssignments (
    AssignmentID INT PRIMARY KEY,
    CustomerID INT,
    PolicyID INT,
    AgentID INT,
    StartDate DATE,
    EndDate DATE,
    CONSTRAINT fk_Customer FOREIGN KEY (CustomerID)
        REFERENCES Customers(CustomerID),
    CONSTRAINT fk_Policy FOREIGN KEY (PolicyID)
        REFERENCES Policies(PolicyID),
    CONSTRAINT fk_Agent FOREIGN KEY (AgentID)
        REFERENCES Agents(AgentID)
);

CREATE TABLE Claims (
    ClaimID INT PRIMARY KEY,
    AssignmentID INT,
    ClaimDate DATE,
    ClaimAmount DECIMAL(10,2),
    ClaimStatus VARCHAR(50),
    CONSTRAINT fk_Assignment FOREIGN KEY (AssignmentID)
        REFERENCES PolicyAssignments(AssignmentID)
);

```

4. Insert Records into each table:

```

--Insert records into Customers
--Insert records into Claims
INSERT INTO Customers(CustomerID, FirstName, LastName, DateOfBirth, Phone, Email)
VALUES
(1, 'Himapriya', 'Pothuganti', '2005-03-20', '833807888', 'himapriya09@gmail.com'),
(2, 'Aarav', 'Sharma', '1998-07-12', '9876543210', 'aarav@gmail.com'),
(3, 'Sneha', 'Iyer', '2002-04-05', '9000012345', 'sneha@gmail.com'),
(4, 'Kiran', 'Reddy', '1999-01-18', '9888899999', 'kiran@gmail.com'),
(5, 'Vikas', 'Kumar', '2004-11-28', '9888899988', 'vikas@gmail.com');

--Insert records into Policies
INSERT INTO Policies (PolicyID, PolicyName, PolicyType, PremiumAmount,
DurationYears) VALUES
(101, 'Life Secure', 'Life', 50000, 20),
(102, 'Health Plus', 'Health', 30000, 10),
(103, 'Motor Protect', 'Motor', 25000, 1),
(104, 'Future Invest', 'Investment', 60000, 20);

--insert records into Agents
INSERT INTO Agents (AgentID, AgentName, Phone, City) VALUES
(201, 'Ravi Kumar', '9123456789', 'Hyderabad'),
(202, 'Neha Singh', '9012345678', 'Bangalore'),
(203, 'Amit Verma', '9888877777', 'Delhi'),
(204, 'Priya Nair', '9555566666', 'Kochi');

--Insert records into PolicyAssignments
INSERT INTO PolicyAssignments
(AssignmentID, CustomerID, PolicyID, AgentID, StartDate, EndDate) VALUES
(301, 1, 101, 201, '2023-01-01', '2043-01-01'),
(302, 2, 102, 202, '2022-06-15', '2032-06-15'),

```

```
(303, 3, 103, 203, '2024-02-01', '2025-02-01'),
(304, 4, 104, 204, '2023-05-10', '2043-05-10');
```

--Insert records into Claims

```
INSERT INTO Claims
(ClaimID, AssignmentID, ClaimDate, ClaimAmount, ClaimStatus)
VALUES
(401, 301, '2024-05-10', 15000.00, 'Approved'),
(402, 302, '2023-11-20', 8000.00, 'Pending'),
(403, 303, '2024-09-10', 55000.00, 'Rejected'),
(404, 304, '2024-10-15', 12000.00, 'Approved');
```

5. Select commands:

Simple Select Commands:

-- 1. View all records from the Customers table.

```
select * from Customers;
```

-- 2. Display only FirstName and LastName from Customers.

```
select FirstName, LastName from Customers;
```

-- 3. Display CustomerID, PolicyID, StartDate and EndDate from PolicyAssignments.

```
select CustomerID, PolicyID, StartDate, EndDate from PolicyAssignments;
```

Operators:

-- 1. Display all policies having premium more than 40000.

```
select * from Policies where PremiumAmount > 40000;
```

-- 2. Display policies of type Health or Life.

```
select * from Policies where PolicyType = 'Health' or PolicyType = 'Life';
```

-- 3. Display policies of type Life, Health, Motor using in operator.

```
select * from Policies where PolicyType in ('Life', 'Health', 'Motor');
```

-- 4. Display policies with PremiumAmount between 20000 and 60000.

```
select * from Policies
where PremiumAmount between 20000 and 60000;
```

WildCards:

-- 1. Display agents whose city second letter is 'a'.

```
select * from Agents where City like '_a%';
```

-- 2. Display customers whose email contains 'gmail' and born after 1998-07-12.

```
select * from Customers where DateOfBirth > '1998-07-12' and Email like
'%gmail%';
```

OrderBy :

- 1. Display customers ordered by DateOfBirth.

```
select * from Customers order by DateOfBirth;
```
- 2. Display agents ordered by City and AgentName.

```
select * from Agents order by City, AgentName;
```
- 3. Display policies ordered by PremiumAmount descending.

```
select * from Policies order by PremiumAmount desc;
```

Aggregate Functions :

- 1. Display highest and lowest claim amount.

```
select min(ClaimAmount) as minimum_claim_amount, max(ClaimAmount) as maximum_claim_amount from Claims;
```
- 2. Display number of rejected claims.

```
select count(*) as no_of_claims_rejected from Claims where ClaimStatus = 'Rejected';
```
- 3. Display total claim amount.

```
select sum(ClaimAmount) as total_claim_amount from Claims;
```
- 4. Display average policy premium.

```
select avg(PremiumAmount) as average_premium from Policies;
```
- 5. Display latest claim record.

```
select top 1 * from Claims order by ClaimDate desc;
```

Date Functions :

- 1. Display all active policies (where today is between StartDate and EndDate).

```
select * from PolicyAssignments where getdate() between StartDate and EndDate;
```
- 2. Display policies that will expire within the next 6 months.

```
select * from PolicyAssignments where EndDate between getdate() and dateadd(month, 6, getdate());
```
- 3. Display number of days left for each policy to expire.

```
select AssignmentID, datediff(day, getdate(), EndDate) as days_remaining from PolicyAssignments;
```
- 4. Display claims raised in the current year.

```
select * from Claims where year(ClaimDate) = year(getdate());
```

String Functions :

-- 1. Convert customer names to uppercase for reporting.

```
select CustomerID, upper(FirstName) as FirstNameUpper, upper(LastName) as LastNameUpper
from Customers;
```

-- 2. Extract first 4 characters of each policy name.

```
select PolicyID, substring(PolicyName, 1, 4) as ShortPolicyName
from Policies;
```

-- 3. Find the length of each customer email.

```
select CustomerID, len(Email) as EmailLength
from Customers;
```

-- 4. Replace part of PolicyType for formatting.

```
select PolicyID, replace(PolicyType, 'Life', 'LIFE') as ModifiedPolicyType
from Policies;
```

Group By ,Having :

-- 1. Display number of policies for each PolicyType.

```
select PolicyType, count(*) as PolicyCount
from Policies
group by PolicyType;
```

-- 2. Display PolicyTypes having more than one policy.

```
select PolicyType, count(*) as PolicyCount
from Policies
group by PolicyType
having count(*) > 1;
```

6.Joins:

-- 1. Display FirstName, PolicyName, AgentName, StartDate and EndDate from their respective tables.

```
select c.FirstName, p.PolicyName, a.AgentName, pa.StartDate, pa.EndDate
from PolicyAssignments pa
join Customers c on pa.CustomerID = c.CustomerID
join Policies p on pa.PolicyID = p.PolicyID
join Agents a on pa.AgentID = a.AgentID;
```

-- 2. Display records of Customers with or without Policies.

```
select c.FirstName, p.PolicyName from Customers c
left join PolicyAssignments pa on c.CustomerID = pa.CustomerID
left join Policies p on pa.PolicyID = p.PolicyID;
```

-- 3. Display all Customers with NO Claims.

```
select c.FirstName, c.LastName from Customers c
left join PolicyAssignments pa on c.CustomerID = pa.CustomerID
left join Claims cl on pa.AssignmentID = cl.AssignmentID
```

```
where cl.ClaimID is null;
```

-- 4. Show names and total claim amount of Customers

```
with Claim Amount > 50000 (Use HAVING Clause).
select c.FirstName, sum(cl.ClaimAmount) as TotalClaimAmount from Customers
c
join PolicyAssignments pa on c.CustomerID = pa.CustomerID
join Claims cl on pa.AssignmentID = cl.AssignmentID
group by c.FirstName
having sum(cl.ClaimAmount) > 50000;
```

-- 5. Display list with Agent Wise Policy Count.

```
select a.AgentName, count(pa.PolicyID) as PolicyCount from Agents a
join PolicyAssignments pa on a.AgentID = pa.AgentID
group by a.AgentName;
```

7.Subqueries :

-- 1. Find customers who have taken premium above average.

```
select * from Customers c
join PolicyAssignments pa on c.CustomerID=pa.CustomerID
join Policies po on po.PolicyID=pa.PolicyID
where po.PremiumAmount=(select avg(PremiumAmount) as av from Policies);
```

-- 2. Policies having highest premium.

```
select * from Customers c
join PolicyAssignments pa on c.CustomerID=pa.CustomerID
join Policies po on po.PolicyID=pa.PolicyID
where po.PremiumAmount=(select max(PremiumAmount) as av from Policies);
```

-- 3. Policies taken by more than one customer.

```
select po.PolicyID, po.PolicyName
from PolicyAssignments pa
join Policies po on po.PolicyID = pa.PolicyID
group by po.PolicyID, po.PolicyName
having count(distinct pa.CustomerID) > 1;
```

-- 4. Find policies with premium greater than ANY premium of policies held by CustomerID = 101.

```
select * from Policies
where PremiumAmount > any (
select p.PremiumAmount from Policies p
join PolicyAssignments pa on p.PolicyID = pa.PolicyID
where pa.CustomerID = 101
);
```

-- 5. Find agents who serve customers from the same city.

```
select distinct a.AgentID, a.AgentName, a.City
from Agents a
where exists (
select 1
from PolicyAssignments pa
join Customers c on pa.CustomerID = c.CustomerID
where pa.AgentID = a.AgentID and c.City = a.City
```

```
);
```

8. Set Operations:

-- 1. Customers who bought Life OR Health policies.

```
select CustomerID
from PolicyAssignments pa
join Policies p on pa.PolicyID = p.PolicyID
where p.PolicyType = 'Life'

union

select CustomerID
from PolicyAssignments pa
join Policies p on pa.PolicyID = p.PolicyID
where p.PolicyType = 'Health';
```

-- 2. Customers who bought both Life AND Health.

```
select CustomerID
from PolicyAssignments pa
join Policies p on pa.PolicyID = p.PolicyID
where p.PolicyType = 'Life'

intersect

select CustomerID
from PolicyAssignments pa
join Policies p on pa.PolicyID = p.PolicyID
where p.PolicyType = 'Health';
```

-- 3. Customers who bought Life but NOT Health.

```
select CustomerID
from PolicyAssignments pa
join Policies p on pa.PolicyID = p.PolicyID
where p.PolicyType = 'Life'

except

select CustomerID
from PolicyAssignments pa
join Policies p on pa.PolicyID = p.PolicyID
where p.PolicyType = 'Health';
```

9. Rollup commands:

-- 1. Display total claim amount by ClaimStatus with grand total using rollup.

```
select ClaimStatus, sum(ClaimAmount) as TotalAmount from Claims
group by rollup(ClaimStatus);
```

-- 2. Display total premium by PolicyType with overall total using rollup.

```
select PolicyType, sum(PremiumAmount) as TotalPremium
from Policies
group by rollup(PolicyType);
```

10. Cube commands:

-- 1. Display total premium by PolicyType and DurationYears with all combinations using cube.

```
select PolicyType, DurationYears, sum(PremiumAmount) as TotalPremium
from Policies
group by cube(PolicyType, DurationYears);
```

-- 2. Display claim totals by ClaimStatus and year using cube.

```
select ClaimStatus, year(ClaimDate) as ClaimYear, sum(ClaimAmount) as
TotalAmount
from Claims
group by cube(ClaimStatus, year(ClaimDate));
```

11. Grouping Sets commands :

-- 1. Display total premium by PolicyType and by DurationYears using grouping sets.

```
select PolicyType, DurationYears, sum(PremiumAmount) as TotalPremium
from Policies
group by grouping sets (
(PolicyType),(DurationYears));
```

-- 2. Display total claim amount by ClaimStatus and by ClaimYear using grouping sets.

```
select ClaimStatus, year(ClaimDate) as ClaimYear, sum(ClaimAmount) as
TotalAmount
from Claims
group by grouping sets (
ClaimStatus),(year(ClaimDate));
```

12. Merge Commands:

-- 1. Merge updated claim amounts from TempClaims table into Claims table.

```
merge Claims as target
using TempClaims as source
on target.ClaimID = source.ClaimID
when matched then
    update set target.ClaimAmount = source.ClaimAmount
when not matched then
    insert (ClaimID, AssignmentID, ClaimDate, ClaimAmount, ClaimStatus)
    values (source.ClaimID, source.AssignmentID, source.ClaimDate,
source.ClaimAmount, source.ClaimStatus);
```

-- 2. Merge updated policy premiums from TempPolicies table into Policies table.

```
merge Policies as target
using TempPolicies as source
on target.PolicyID = source.PolicyID
when matched then
    update set target.PremiumAmount = source.PremiumAmount
when not matched then
    insert (PolicyID, PolicyName, PolicyType, PremiumAmount,
DurationYears)
    values (source.PolicyID, source.PolicyName, source.PolicyType,
source.PremiumAmount, source.DurationYears);
```